Workshop – Week 8 – Some selected solutions

1. Run the program in the notes that uses the recursive factorial function. Enter the number 24. Explain what happens.

Solution to Q1:

Factorials get very big. If the number goes past the maximum integer (maxint) that you can store in a computer, then a 1 will be carried into the sign bit and a random negative result will appear instead of a very large positive number. It is extremely important for programmers to be aware of maxint and the effect it can have. Usually this type of problem does not produce an error.

2. In the program that uses the recursive factorial function, modify the function as follows:

```
int factorial(int x) {
  int result;
/**
  if (x == 0) {
    return 1;    // the base case
  }
**/
  cout << x << ", ";
  result = x * factorial(x - 1);
  return result;
}
```

Now run the program for a value of 5. Explain what happens.
Basically, we removed the base case. This should cause a stack overflow but some systems look out for this and stop the program before the stack overflow occurs.

Solution to Q2:

Without the base case the programs runs forever or until the stack fills up (keeping information about each function call).

3. You have won first place in a competition and you now get to select 2 prizes from a group of 6 prizes. How many different combinations of 2 prizes could you possibly select?
We use the formula `C(n, r)` where `C` stands for "Combinations", `n` is the total number of items and `r` is the number of items you wish to select. The actual formula is:

```
C(n, r) = n! / (r! * (n – r)!)
```

Write a program that uses this formula (along with the recursive factorial function) to calculate how many different combinations of 2 prizes are possible with a total number of 6 prizes.
The correct answer is 15

Solution to Q3:

```
#include <iostream>
using namespace std;
```

```
int factorial(int x);

int main() {
  int combination, temp, n, r;
  cout << "How many prizes are there? ";
  cin >> n;
  cout << "How many prizes can you select? ";
  cin >> r;
  temp = factorial(r) * factorial(n - r);
  combination = factorial(n) / temp;
  cout << "The number of combinations is " << combination << endl;
}

int factorial(int x) {
  int result;
  if (x == 0) {
    return 1;    // the base case
  }
  result = x * factorial(x - 1);
  return result;
}
```

4. There are two ways to compare the speed of algorithms. One way is to time the program but this is not always accurate. A program may pause if the system needs to do urgent work, e.g. a security check. Another way to compare algorithms is to count important operations. For example, when looking at search algorithms, **the important operations are the comparisons**. In other words, how often do we need to ask "is this item in the array the one that I am searching for?".

Assume an array of 10,000 numbers in order from 1 to 10,000. How many comparisons are required to find the number 2,500 if:

a) you are using linear search?

b) you are using binary search?

Solution to Q4:

There are 10,000 numbers stored in array so the array looks like this:

0	1	2	3
1	2	3	4

... and so on up to

9997	9998	9999
9998	9999	10000

Solution to Q4(a):

Linear search runs through the array checking each value, one after the other. To find the number 2,500 the algorithm would check element[0] (1 comparison), then element[1] (2 comparisons), then element[2] (3 comparisons) and so on until it checks element[2499] and finds the number 2,500. Thus the linear search algorithm would require 2,500 comparisons to find the number 2,500.

Solution to Q4(b):

Binary search looks at the first index (0) and the last index (9999) and calculates the middle index which would be (0 + 9999) / 2 = 4999. Then it would look at element[4999] and find the number 5,000

which is greater than 2,500 (the number we are searching for). It would then make a recursive function call to do binary search with a first index of 0 (unchanged) and a last index of 4998 (middle – 1). The new function call would calculate the middle index to be (0 + 4998) / 2 = 2499. Then it would look at element[2499] and find the number 2500 and would stop because we have found the number we were searching for. Thus the binary search algorithm would need 2 comparisons to find the number 2500.

Conclusion:

The binary search algorithm is significantly faster than the linear search algorithm AS LONG AS the data has been sorted. Note that we can deduce which algorithm is faster without running the program. This is an important part of Computer Science – being able to predict which algorithm is more efficient before you actually write the program.

5. Design your own system of caves and prepare to use the caves program from the notes. You will need to do the following:

1. Draw a map of some caves with doors that lead to other caves (similar to the map in the notes).
2. Write the `loadalldata` function for your system of caves.
3. Run the program.

Solution to Q5:

Draw a map of your own system of caves – use the example in the notes for ideas. Put some interesting links into it. Create a line of code (to go inside loadalldata) for each cave. For example, if cave 15 links to caves 9, 12 and 7 (the order does not matter) then the line in loadalldata would be:

```
cave[15].loaddata(3, 9, 12, 7, -1, -1);
```

The first number is the number of doors leading from the cave. Start your system from cave[0].